CSCI 611: Applied Machine Learning

Sai Rikwith Daggu [012147396]

Assignment#4-

In this report, we explore style transfer, a deep learning technique that applies the artistic style of one image (such as a painting) to the content of another (like a photograph). Our approach utilizes a pretrained VGG19 network to merge the content of one image with the artistic characteristics of another. The implementation follows the Neural Style Transfer (NST) method introduced in the paper "A Neural Algorithm of Artistic Style" by Gatys et al. (2015). This technique extracts content and style features from a convolutional neural network and optimizes a target image to align with both.

To Do: mapping of layer names to the names

```python
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al (2016)
    """
    # Mapping of layer names in PyTorch VGGNet to layers from the paper
    if layers is None:
        layers = {
            '0': 'conv1_1',
            '5': 'conv2_1',
            '10': 'conv3_1',
            '19': 'conv4_1',
            '21': 'conv4_2',  # content representation
            '28': 'conv5_1'
        }

    features = {}
    x = image
    # Model's layers
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

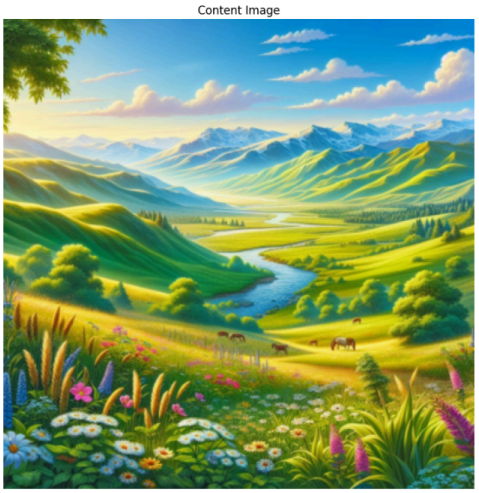
To Do: Gram Matrix

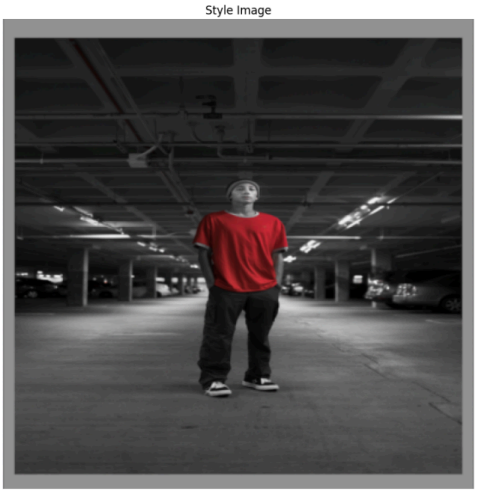
```python
[23] def gram_matrix(tensor):
        """ Calculate the Gram Matrix of a given tensor
            Gram Matrix: https://en.wikipedia.org/wiki/Gramian_matrix
        """
        # Get the batch_size, depth, height, and width of the tensor
        _, d, h, w = tensor.size()

        # Reshape the tensor to flatten spatial dimensions
        tensor = tensor.view(d, h * w)

        # Calculate the gram matrix by multiplying the reshaped tensor by its transpose
        gram = torch.mm(tensor, tensor.t())

        return gram
```

To Do: Updating the Target & Calculating Losses

```
[26] # for displaying the target image, intermittently
     show_every = 400

     # iteration hyperparameters
     optimizer = optim.Adam([target], lr=0.003)
     steps = 2000  # decide how many iterations to update your image (5000)

     for ii in range(1, steps + 1):
         ## -- get the features from your target image -- ##
         target_features = get_features(target, vgg)

         # Calculate the content loss
         content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2']) ** 2)

         # The style loss
         style_loss = 0
         # iterate through each style layer and add to the style loss
         for layer in style_weights:
             # get the "target" style representation for the layer
             target_feature = target_features[layer]
             _, d, h, w = target_feature.shape

             # Calculate the target gram matrix
             target_gram = gram_matrix(target_feature)

             # get the "style" style representation for the layer
             style_feature = style_features[layer]

             # Calculate the style gram matrix
             style_gram = style_grams[layer]

             # Calculate the style loss for one layer, weighted appropriately
             layer_style_loss = torch.mean((target_gram - style_gram) ** 2)

             # Add to the total style loss, normalized by size
             style_loss += layer_style_loss / (d * h * w)

         # Calculate the total loss
         total_loss = content_weight * content_loss + style_weight * style_loss

         ## -- do not need to change code, below -- ##
         # update your target image
         optimizer.zero_grad()
         total_loss.backward()
         optimizer.step()

         # display intermediate images and print the loss
         if ii % show_every == 0:
             print(f"Step {ii}/{steps}, Total Loss: {total_loss.item()}")
             plt.imshow(im_convert(target))
             plt.show()
```

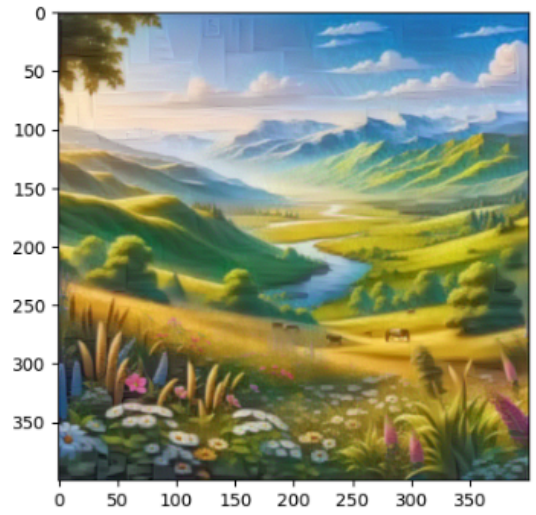
Content Image and Style image



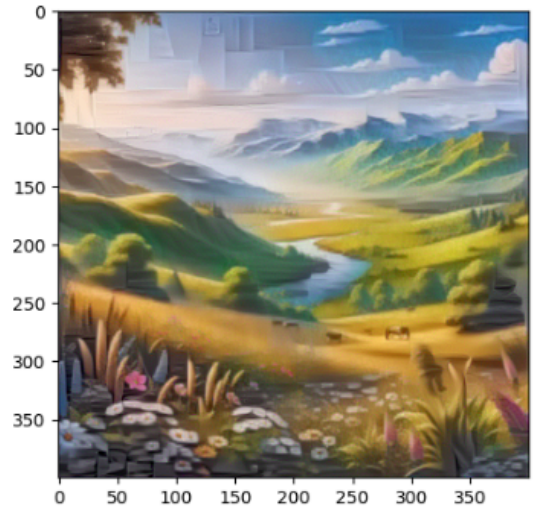


Visual Progress:

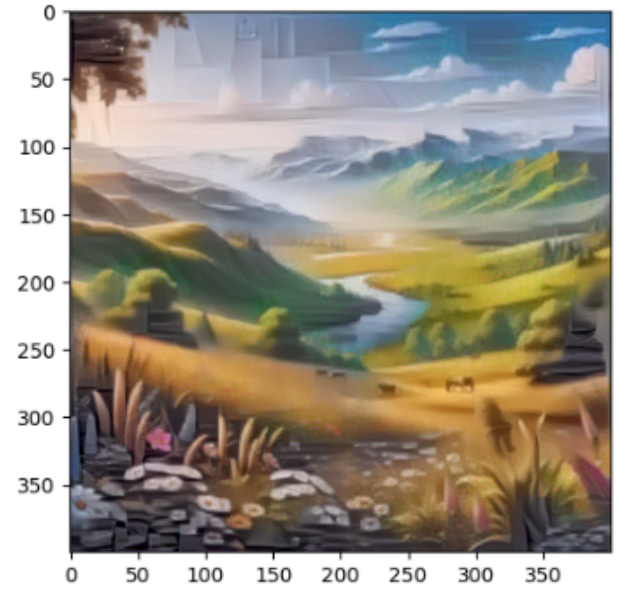
Step 400/2000, Total Loss: 89645200.0



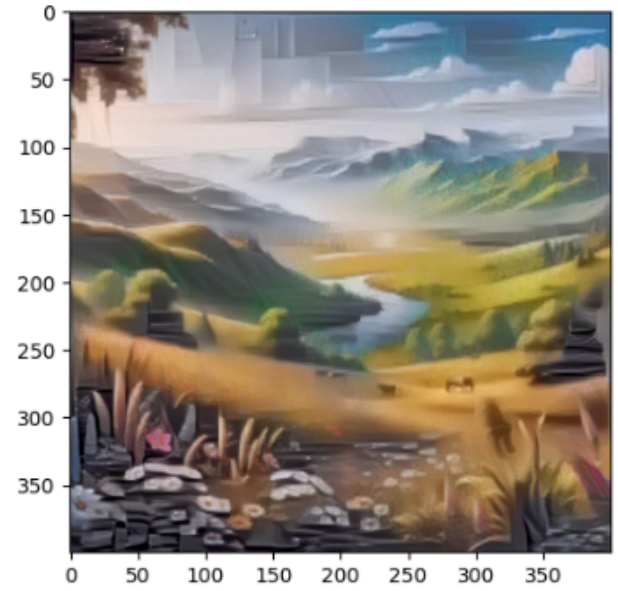
Step 800/2000, Total Loss: 36055336.0



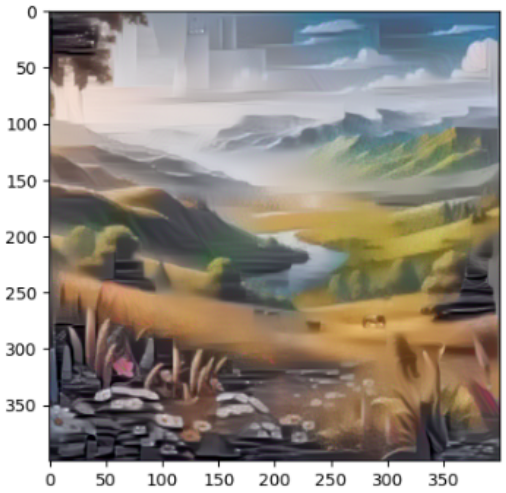
Step 1200/2000, Total Loss: 21679152.0



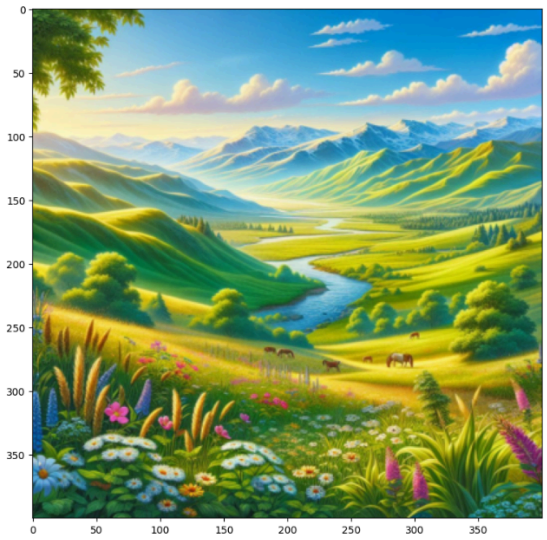
Step 1600/2000, Total Loss: 15438686.0



Step 2000/2000, Total Loss: 11958958.0



Final Target Image:



Observations: The image gradually transformed, blending strong abstract brush strokes from the style image. Loss decreased over time showing convergence.

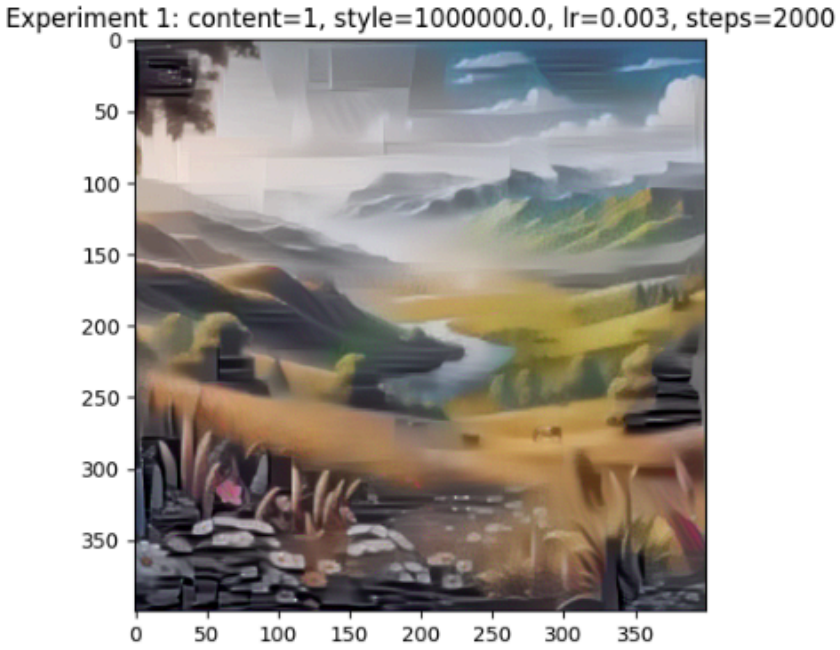
Hyperparameter Experiments:

I conducted 3 experiments by varying key hyperparameters to understand their impact on the style transfer results.

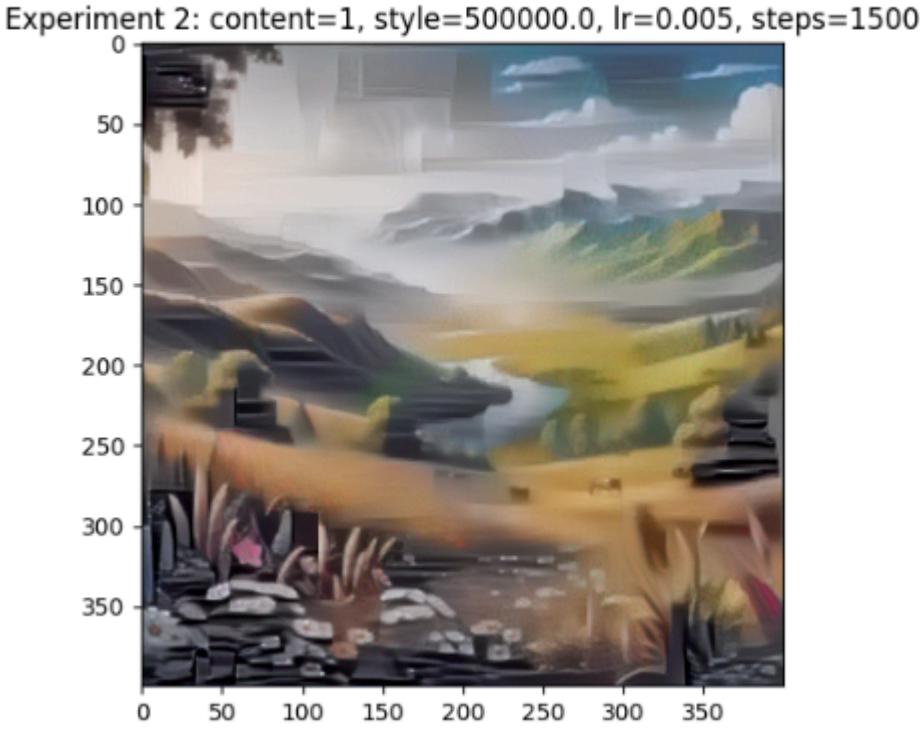
```
# Define different hyperparameter settings
hyperparams = [
    {'content_weight': 1, 'style_weight': 1e6, 'lr': 0.003, 'steps': 2000},
    {'content_weight': 1, 'style_weight': 5e5, 'lr': 0.005, 'steps': 1500},
    {'content_weight': 5, 'style_weight': 1e7, 'lr': 0.001, 'steps': 2500}
]

# Iterate over different hyperparameter settings
for idx, params in enumerate(hyperparams):
    print(f"\nExperiment {idx+1}: content_weight={params['content_weight']}, "
          f"style_weight={params['style_weight']}, lr={params['lr']}, steps={params['steps']}")

    content_weight = params['content_weight']
    style_weight = params['style_weight']
    lr = params['lr']
    steps = params['steps']

    # Initialize target image
    target = content.clone().requires_grad_(True).to(device)

    # Define optimizer
    optimizer = optim.Adam([target], lr=lr)
```

```python
# Training Loop
for ii in range(1, steps+1):
    target_features = get_features(target, vgg)

    # Content loss
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)

    # Style loss
    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]
        layer_style_loss = torch.mean((target_gram - style_gram)**2)

        style_loss += style_weights[layer] * layer_style_loss / (d * h * w)

    # Total loss
    total_loss = content_weight * content_loss + style_weight * style_loss

    # Update target image
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    # Print loss every few steps
    if ii % 400 == 0:
        print(f"Step {ii}: Total Loss: {total_loss.item()}")

# Display final output for this experiment
plt.figure(figsize=(10, 5))
plt.imshow(im_convert(target))
plt.title(f"Experiment {idx+1}: content={content_weight}, style={style_weight}, lr={lr}, steps={steps}")
plt.show()
```

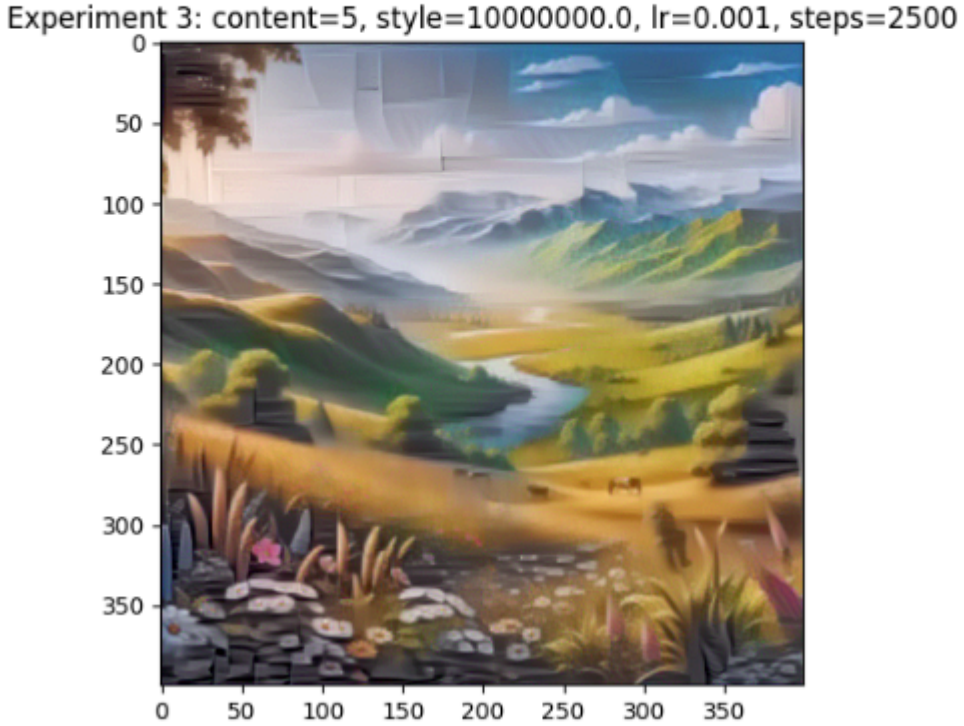
```
Experiment 1: content_weight=1, style_weight=1000000.0, lr=0.003, steps=2000
Step 400: Total Loss: 54815028.0
Step 800: Total Loss: 22257244.0
Step 1200: Total Loss: 13690198.0
Step 1600: Total Loss: 9895304.0
Step 2000: Total Loss: 7779728.0
```



```
Experiment 2: content_weight=1, style_weight=500000.0, lr=0.005, steps=1500
Step 400: Total Loss: 14542165.0
Step 800: Total Loss: 6450708.5
Step 1200: Total Loss: 4240864.5
```




```
Experiment 3: content_weight=5, style_weight=10000000.0, lr=0.001, steps=2500
Step 400: Total Loss: 1720189312.0
Step 800: Total Loss: 862329920.0
Step 1200: Total Loss: 509162912.0
Step 1600: Total Loss: 336966304.0
Step 2000: Total Loss: 243404576.0
Step 2400: Total Loss: 187129680.0
```



Findings:
Higher style weights increase artistic texture but can distort content. More steps improve convergence and detail. Visual outputs show the best results with a balanced style weight and enough steps.

Conclusion:
This implementation successfully reproduced the neural style transfer algorithm described by Gatys et al. The experiments underscored the significant impact of hyperparameter selection, as each parameter played a distinct role in shaping the stylization process.

The results demonstrate how deep neural networks can effectively separate and merge content and style representations, producing unique artistic images that blend the structure of one image with the aesthetic characteristics of another.

The best outcomes were achieved with a moderate style weight (1e6), a layer weighting strategy that slightly prioritized earlier layers, around 2500 iterations, and a learning rate of 0.001.